

Table of Contents

1	Introduction	3
1.1	Purpose.....	3
1.2	Scope	3
1.3	Overview.....	3
2	The Overall Description	3
2.1	Application Software Perspective.....	3
2.1.1	System Interfaces	4
2.1.2	User Interfaces	4
2.1.3	Hardware Interfaces	4
2.1.4	Software Interfaces	4
2.1.5	Communications Interfaces	5
2.1.6	Memory Constraints.....	5
2.1.7	Operations	5
2.1.8	Site Adaptation Requirements	5
2.1.9	Application Software Functions	5
2.2	User Characteristics	6
2.3	Constraints	6
2.4	Assumptions and Dependencies	6
2.5	Apportioning of Requirements	6
3	Specific Requirements	6
3.1	External Interfaces	6
3.2	Functional Requirements	7
3.2.1	Signup and login use case.....	7
3.2.2	Website configuration and user roles use case.	7
3.2.3	Feedback ingestion use case.	8
3.2.4	Data preprocessing and preparation use case.....	8
3.2.5	Feedback classification use case.....	8
3.2.6	Prioritization use case.	8
3.2.7	Feedback storage and management use case.	9
3.2.8	Dashboard and analysis use case.	9
3.3	Non-functional requirements	9
3.3.1	Reliability use case.	9

3.3.2	Availability use case	10
3.3.3	Security use case	10
3.3.4	Maintainability use case.....	10
3.3.5	Portability use case	10
3.4	Performance Requirements	10
3.5	Logical Database Requirements	10
3.6	Design Constraints	11
3.6.1	Standards Compliance	11
3.7	Organizing the Specific Requirements by Functional Hierarchy	11
3.8	Additional Comments	12
4	Change Management Process	12
5	Appendices.....	12
5.1	Acronym	12
5.2	References.....	12

1 Introduction

1.1 Purpose

The purpose of this document is to define the logical components of the Artificial Intelligence (AI)-powered Feedback Management System (FMS). This platform is designed to help Small and Medium Enterprises (SMEs) to centralize, detect sentiment and emotions of input comments, assign priority scoring for each complaint and analyze customer complaints from multiple channels e.g., social media. The document serves as a guide for stakeholders, developers, and quality assurance teams.

1.2 Scope

The system provides a centralized website dashboard where company administrators can.

- Integrate social media and messaging Application Programming Interfaces (APIs) (Facebook, WhatsApp, etc.).
- Automatically ingest and transcribe multimodal data e.g., text.
- Utilize AI models for sentiment analysis, emotion detection, problem type.
- Assign priority score based on the outputs of the AI models predictions and business rules.

Benefits of the proposed solution are to help SMEs in handling complaints in an efficient manner by providing insights and Key Performance Indicators (KPIs) to ease the process of handling complaints.

1.3 Overview

This document outlines the system's architecture, user roles, technical constraints, detailed functional and non-functional requirements, user scenarios and technical specifications for the development of the website. It focuses on the transition from raw, unstructured data to actionable business intelligence.

2 The Overall Description

The system is a standalone web-based Software as a Service (SaaS) platform. It acts as an intelligence layer that sits between a company's communication channels and its decision-makers.

2.1 Application Software Perspective

The CMS is a web-based AI-powered platform designed to help small and medium enterprises collect, analyze, and manage customer complaints from multiple data sources such as social media platforms (Facebook, Twitter/X, WhatsApp). The system operates as an independent application but integrates with external systems through APIs provided by social media platforms. It functions as a centralized hub for data collection and intelligent analysis using Natural Language Processing (NLP).

Compared to traditional complaints management systems, this system provides.

- Automated data extraction from multiple sources
- AI-based complaint classification
- Sentiment and emotion analysis
- Priority assignment based on business rules

- Visual dashboards for decision support

2.1.1 System Interfaces

The system will interface with.

- Social media APIs (Facebook Graph API, Twitter/X API, WhatsApp Business API)
- Authentication services
- Database server
- Web browser clients (admin users)

2.1.2 User Interfaces

The system provides a web-based Graphical User Interface (GUI) with the following components.

- Login and registration pages
- API configuration page
- Domain selection page (e.g., food, e-commerce)
- Complaint dashboard
- Complaint list and details page
- Analytics and reports page
- Settings and profile management page
- Interface characteristics.
 - Responsive web design
 - Dashboard with charts and statistics
 - Filter and search options
 - Simple navigation menu
 - Multi-language support

2.1.3 Hardware Interfaces

The system has no direct hardware interface requirements. It interacts only with standard computing devices such as.

- Web servers
- Client computers
- Mobile devices (via browser)

2.1.4 Software Interfaces

Required external software and APIs include.

1. Database management system
 - **Name.** Relational database (e.g., MySQL/PostgreSQL)
 - **Purpose.** Store complaints, users, and analysis results
2. Social media APIs
 - Facebook graph API
 - Twitter/X API
 - WhatsApp business API
3. AI services
 - Sentiment analysis API
 - Emotion classification API

- Problem domain classification API
4. Web server
- Application server for hosting the system

2.1.5 Communications Interfaces

The system will be used.

- HTTPS protocol for secure communication
- RESTful APIs for data exchange
- JSON format for data transmission
- TCP/IP network protocol

2.1.6 Memory Constraints

No strict memory constraints are imposed. The system should be able to operate on standard cloud or enterprise servers with sufficient memory to handle.

- Complaint data storage
- AI model execution
- Dashboard analytics

2.1.7 Operations

The system supports.

- Batching data collection from APIs
- Complaint preprocessing and prediction
- Monthly data backups
- Error logging and recovery
- User-initiated operations (viewing dashboards, exporting reports)

2.1.8 Site Adaptation Requirements

Each organization must.

- Register its own API credentials
- Configure complaint domains
- Define business rules for priority levels (optional)
- Set preferred language and categories

2.1.9 Application Software Functions

Major functions include.

- User registration and authentication
- API configuration and management
- Data extraction from social platforms
- Complaint storage and preprocessing
- Complaint classification by domain
- Sentiment and emotion analysis
- Priority scoring
- Dashboard visualization
- Reporting and exporting data

2.2 User Characteristics

Intended users.

- Company administrators
- Business managers
- Customer service supervisors

User profile.

- Basic computer literacy
- No technical AI knowledge required
- Experience with business dashboards (preferred)

2.3 Constraints

Constraints include.

- Dependency on third-party APIs availability
- Internet connectivity requirement
- Data privacy regulations
- Limited API request quotas

2.4 Assumptions and Dependencies

Assumptions.

- Users have valid API keys
- External APIs remain operational
- Users provide correct configuration
- Cloud hosting environment available (preferred)

Dependencies.

- Social media platform policies
- AI service uptime
- Database server stability

2.5 Apportioning of Requirements

Phase 1

- Social media integration
- Complaint preprocessing

Phase 2

- Text prediction (emotion, sentiment and problem type)
- Priority scoring engine

Phase 3

- Dashboard and predictive insights

3 Specific Requirements

3.1 External Interfaces

Input Interfaces.

- Social media comments

- Manual complaint entry

Output Interfaces.

- Dashboard charts
- Complaint reports
- CSV/PDF exports

Data format: JSON

Timing: Real-time or scheduled sync

3.2 Functional Requirements

3.2.1 Signup and login use case.

Brief description. Ensure a seamless process for users to sign up or login without any hassle.

Initial step-by-step description.

For manager.

1. If manager doesn't have an account and enters the website for the first time, the manager signs up.
2. In this process the manager must enter an email address, or a unique username (the system will alert if the username is not unique).
3. The manager must enter a strong password that contains at least eight characters with a mix of uppercase, lowercase, number and symbols.
4. He must reconfirm the entered password in a different field.
5. The manager then defines his name, the company's name, company's domain (e.g., e-commerce, food), complaints' categories (for food domain: food quality, service quality, hygiene, etc....).
6. The system allows the manager to enter API keys from any social platform (he must at least enter one API key) and if he doesn't know how, the system can direct him on how the process goes.
7. The manager then logs on to the website.
8. The system then requires login credentials verification for validation.

For Customer Service Supervisor (CSS).

1. CSS logs in with the provided account from manager via username and password.

For tech admin.

1. The tech admin logs in with the provided account from manager via username and password.

3.2.2 Website configuration and user roles use case.

Brief description. Defining each user role (manager, CSS, tech admin), how the website is configured and by which user.

Initial step-by-step description.

User Roles

- **Manager** has all permissions such as viewing the feedback database, viewing dashboard and configure website settings in terms API configurations, manipulating user roles (if there is more than one user), etc....
- **CSS** has some permissions such as viewing the feedback database and viewing dashboard only without taking part in configuring website settings.
- **Tech admin** has only one job which is managing user roles and website settings

configurations.

Website configurations

- **Credentials and common settings**
 1. Manager or tech admin can configure credentials and settings.
 2. The system saves the configuration correctly and syncs it up across users.
- **API configurations**
 1. They can also configure API keys for platforms like Facebook, Twitter/X, WhatsApp.
 2. The system ensures the provided API is correct before saving.
- **Managing user roles**
 1. Finally, they can manage user roles for website users.
 2. Whether to edit a user's role or delete the user entirely.
 3. After the changes are made the system saves them correctly.

3.2.3 Feedback ingestion use case.

Brief description. Automatically gather customer feedback from different channels.

Initial step-by-step description.

1. Manager or CSS chooses the database tab.
2. The system automatically uses API calls to fetch messages or feedback.
3. Fetch can be scheduled (e.g., every hour).
4. The system validates the structure of the incoming feedback.
5. The system saves the feedback in the database with a unique identifier.
6. The system forwards for further processing.

3.2.4 Data preprocessing and preparation use case

Brief description. After the previous use case, step 6 is implemented. The system prepares feedback content for automated analysis.

Initial step-by-step description.

1. The system handles multilingual feedback inputs by routing it to the appropriate pipeline.
2. The system ensures feedback text consistency and cleanliness using tokenization, stop words removal and lemmatization.
3. Finally, the system forwards the prepared feedback data for analytical processes.

3.2.5 Feedback classification use case.

Brief description. After the previous use case 3.2.4 is implemented, the system organizes the feedback into categories relevant to the company's business. Determines the general sentiment of the feedback such as positive, neutral, or negative. Identifies the emotion expressed in the feedback such as frustrated, neutral, disgusted and satisfied.

Initial step-by-step description.

1. The system uses machine learning or deep learning classification.
2. Each feedback is classified automatically with its domain category.
3. Each feedback is tagged with a sentiment category or label.
4. Each feedback is tagged with an emotion category or label

3.2.6 Prioritization use case.

Brief description. After the previous use case 3.2.5 is implemented, the system decides which feedback needs immediate attention based on general or manager's business rules.

Initial step-by-step description.

1. The manager can either use a general rule prioritization or assign prioritization based on his business rules.
2. The system combines sentiment, emotion, and domain problem to calculate priority (Critical, High, Medium and Low).
3. Rules example.
 - Negative sentiment + Frustration emotion + Delivery Issue = High priority
 - Negative sentiment + Neutral emotion + Hygiene = Medium priority

3.2.7 Feedback storage and management use case.

Brief description. After the previous use cases 3.2.4, 3.2.5, 3.2.6 are implemented, the system keeps all feedback data safe and organized for a visualized analysis.

Initial step-by-step description.

1. The system stores raw customers' feedback and processed results (sentiment, domain problem, emotion, priority) in relational database tables.
2. The system maintains timestamps and ID for each feedback.
3. The manager or CSS can view the database.
4. The manager or CSS can edit the status of each feedback (complaint) (open, closed or resolved).
5. The system enables the manager or CSS to filter the database based on a few factors such as priority level, status, sentiment, etc....

3.2.8 Dashboard and analysis use case.

Brief description. The system provides a visual, easy-to-understand overview of customers' feedback.

Initial step-by-step description.

1. The manager or CSS chooses the dashboard tab.
2. The system displays a fully visualized dashboard.
3. The dashboard shows.
 - Number of complaints by domain problem
 - Sentiment distribution (pie chart)
 - Priority breakdown (bar chart)
 - Latest complaints (table)
 - Filters to view by date, platform, priority, or domain.
4. The manager or CSS chooses the *export report* button
5. The system provides a couple of export options e.g., excel, pdf and csv.

3.3 Non-functional requirements

3.3.1 Reliability use case.

Brief description. System shall recover from failures without data loss as it will be hosted locally with low fault tolerance.

Initial step-by-step description.

1. The system saves data on an external device periodically.
2. If any failure or power outage occurs the system will recover the data from the external device.

3. The external device will then be updated on the new data.

3.3.2 Availability use case

Brief description. System shall be available 24/7 with 99% uptime.

Initial step-by-step description.

1. The system will be working 24 hours a day and 7 days a week, meaning any user can enter the website at different times to his/hers liking.
2. If there is maintenance needed the website will announce when it's going to be down and it will be down for a couple of minutes to ensure availability.

3.3.3 Security use case

Brief description. System will secure each account to prevent any data theft.

Initial step-by-step description.

- The system will use encrypted passwords for better security.
- Role-based access; meaning that each user has different permissions to prevent unauthorized access.
- Secure API storage, to prevent data loss of data ingestion sources.
- The system will audit logs to ensure the working system is not hacked by identifying unusual patterns and resolving them.

3.3.4 Maintainability use case

Brief description. How the system will be maintained after release as the system will be divided into components that integrate seamlessly together for better/easier understanding and updating.

Initial step-by-step description.

- The system will inherit from a modular architecture.
- The system will be clearly documented.
- The system will update regularly based on reviews to ensure ease of use.

3.3.5 Portability use case

Brief description. How the system will be delivered and on what device.

Initial step-by-step description.

- The system will work on major browsers that can be accessed via computers, laptops, tablets and phones.
- The system will also have the option to be deployable on cloud servers.

3.4 Performance Requirements

- Support at least 500 concurrent users
- Process 95% of complaints within 3 seconds
- Handling up to 100,000 complaints per month
- Dashboard loading time less than 2 seconds

3.5 Logical Database Requirements

Entities.

- *Company*
- *User*

- **Role**
- **Domain**
- **Feedback Category**
- **Feedback**
- **API**

Relationships.

- **Company** must belong to at least one **Domain** (many-to-one).
- **Domain** may contain one or more **Company** (one-to-many).
- **Company** must have at least one **User** (one-to-many).
- **Users** must belong to one **Company** (many-to-one).
- **Users** must be assigned only one **Role** (many-to-one).
- **Role** can be assigned to many **Users** (one-to-many).
- **Domain** includes many **Feedback Categories** (one-to-many).
- **Feedback Category** must be included in one **Domain** (many-to-one).
- **Feedback** must be classified based on one **Feedback Category** (many-to-one).
- **Feedback Category** can classify many **Feedback** (one-to-many).
- **API** can generate many **Feedback** (one-to-many).
- **Feedback** must be generated from one **API** (many-to-one).
- **Company** can configure many **APIs** (one-to-many).
- **API** must be configured by only one **Company** (many-to-one).

Data retention.

- Feedbacks are stored for minimum 2 years

3.6 Design Constraints

- Must be web-based
- Must support REST APIs
- Must comply with data privacy laws
- Must use secure authentication

3.6.1 Standards Compliance

- HTTPS security standard
- OAuth authentication
- ISO data protection standards
- Audit logging

3.7 Organizing the Specific Requirements by Functional Hierarchy

Main modules.

- Authentication module
- API integration module
- Data preprocessing module
- AI predictions module
- Dashboard module

Future enhancements.

- ## 4 Change Management Process

- Submitted in written form
- Reviewed by project supervisor
- Approved by development team
- Logged in change history
- Implemented in next iteration
- Version control will be maintained using a repository system.

5.1 Acronym

5.2 References

- 12

- Dr. A. Alshazly, "Applying the Software Process Improvement Guide (SPIG) to the Software Projects," ed, 2025.